3.6 Finite Automata

We shall now discover how Lex turns its input program into a lexical analyzer. At the heart of the transition is the formalism known as *finite automata*. These are essentially graphs, like transition diagrams, with a few differences:

1. Finite automata are *recognizers*; they simply say "yes" or "no" about each possible input string.
2. Finite automata come in two flavors:
   (a) *Nondeterministic finite automata* (NFA) have no restrictions on the labels of their edges. A symbol can label several edges out of the same state, and ϵ, the empty string, is a possible label.
   (b) *Deterministic finite automata* (DFA) have, for each state, and for each symbol of its input alphabet exactly one edge with that symbol leaving that state.

Both deterministic and nondeterministic finite automata are capable of recognizing the same languages. In fact these languages are exactly the same languages, called the *regular languages*, that regular expressions can describe.[4]

3.6.1 Nondeterministic Finite Automata

A *nondeterministic finite automaton* (NFA) consists of:

1. A finite set of states S.
2. A set of input symbols Σ, the *input alphabet*. We assume that ϵ, which stands for the empty string, is never a member of Σ.
3. A *transition function* that gives, for each state, and for each symbol in $\Sigma \cup \{\epsilon\}$ a set of *next states*.
4. A state s_0 from S that is distinguished as the *start state* (or *initial state*).
5. A set of states F, a subset of S, that is distinguished as the *accepting states* (or *final states*).

We can represent either an NFA or DFA by a *transition graph*, where the nodes are states and the labeled edges represent the transition function. There is an edge labeled a from state s to state t if and only if t is one of the next states for state s and input a. This graph is very much like a transition diagram, except:

[4]There is a small lacuna: as we defined them, regular expressions cannot describe the empty language, since we would never want to use this pattern in practice. However, finite automata *can* define the empty language. In the theory, $\emptyset$ is treated as an additional regular expression for the sole purpose of defining the empty language.

a) The same symbol can label edges from one state to several different states, and

b) An edge may be labeled by ϵ, the empty string, instead of, or in addition to, symbols from the input alphabet.

Example 3.14: The transition graph for an NFA recognizing the language of regular expression **(a|b)*abb** is shown in Fig. 3.24. This abstract example, describing all strings of a's and b's ending in the particular string abb, will be used throughout this section. It is similar to regular expressions that describe languages of real interest, however. For instance, an expression describing all files whose name ends in `.o` is **any*** `.o`, where **any** stands for any printable character.

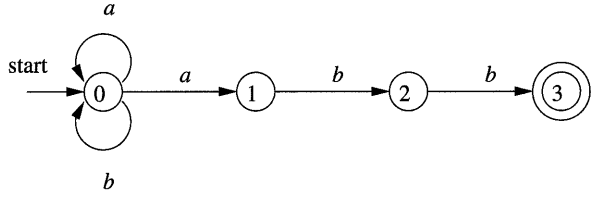


Figure 3.24: A nondeterministic finite automaton

Following our convention for transition diagrams, the double circle around state 3 indicates that this state is accepting. Notice that the only ways to get from the start state 0 to the accepting state is to follow some path that stays in state 0 for a while, then goes to states 1, 2, and 3 by reading abb from the input. Thus, the only strings getting to the accepting state are those that end in abb. □

3.6.2 Transition Tables

We can also represent an NFA by a *transition table*, whose rows correspond to states, and whose columns correspond to the input symbols and ϵ. The entry for a given state and input is the value of the transition function applied to those arguments. If the transition function has no information about that state-input pair, we put $\emptyset$ in the table for the pair.

Example 3.15: The transition table for the NFA of Fig. 3.24 is shown in Fig. 3.25. □

The transition table has the advantage that we can easily find the transitions on a given state and input. Its disadvantage is that it takes a lot of space, when the input alphabet is large, yet most states do not have any moves on most of the input symbols.

STATE	a	b	ϵ
0	$\{0,1\}$	$\{0\}$	$\emptyset$
1	$\emptyset$	$\{2\}$	$\emptyset$
2	$\emptyset$	$\{3\}$	$\emptyset$
3	$\emptyset$	$\emptyset$	$\emptyset$

Figure 3.25: Transition table for the NFA of Fig. 3.24

3.6.3 Acceptance of Input Strings by Automata

An NFA *accepts* input string x if and only if there is some path in the transition graph from the start state to one of the accepting states, such that the symbols along the path spell out x. Note that ϵ labels along the path are effectively ignored, since the empty string does not contribute to the string constructed along the path.

Example 3.16: The string *aabb* is accepted by the NFA of Fig. 3.24. The path labeled by *aabb* from state 0 to state 3 demonstrating this fact is:

$$0 \xrightarrow{a} 0 \xrightarrow{a} 1 \xrightarrow{b} 2 \xrightarrow{b} 3$$

Note that several paths labeled by the same string may lead to different states. For instance, path

$$0 \xrightarrow{a} 0 \xrightarrow{a} 0 \xrightarrow{b} 0 \xrightarrow{b} 0$$

is another path from state 0 labeled by the string *aabb*. This path leads to state 0, which is not accepting. However, remember that an NFA accepts a string as long as *some* path labeled by that string leads from the start state to an accepting state. The existence of other paths leading to a nonaccepting state is irrelevant. □

The *language defined* (or *accepted*) by an NFA is the set of strings labeling some path from the start to an accepting state. As was mentioned, the NFA of Fig. 3.24 defines the same language as does the regular expression **(a|b)*abb**, that is, all strings from the alphabet $\{a, b\}$ that end in *abb*. We may use $L(A)$ to stand for the language accepted by automaton A.

Example 3.17: Figure 3.26 is an NFA accepting $L(\mathbf{aa}^*|\mathbf{bb}^*)$. String *aaa* is accepted because of the path

$$0 \xrightarrow{\varepsilon} 1 \xrightarrow{a} 2 \xrightarrow{a} 2 \xrightarrow{a} 2$$

Note that ϵ's "disappear" in a concatenation, so the label of the path is *aaa*. □

3.6.4 Deterministic Finite Automata

A *deterministic finite automaton* (DFA) is a special case of an NFA where:

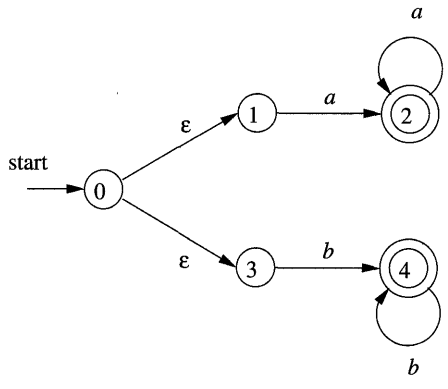


Figure 3.26: NFA accepting $\mathbf{aa^*|bb^*}$

1. There are no moves on input ϵ, and
2. For each state s and input symbol a, there is exactly one edge out of s labeled a.

If we are using a transition table to represent a DFA, then each entry is a single state. we may therefore represent this state without the curly braces that we use to form sets.

While the NFA is an abstract representation of an algorithm to recognize the strings of a certain language, the DFA is a simple, concrete algorithm for recognizing strings. It is fortunate indeed that every regular expression and every NFA can be converted to a DFA accepting the same language, because it is the DFA that we really implement or simulate when building lexical analyzers. The following algorithm shows how to apply a DFA to a string.

Algorithm 3.18: Simulating a DFA.

INPUT: An input string x terminated by an end-of-file character **eof**. A DFA D with start state s_0, accepting states F, and transition function *move*.

OUTPUT: Answer "yes" if D accepts x; "no" otherwise.

METHOD: Apply the algorithm in Fig. 3.27 to the input string x. The function $move(s, c)$ gives the state to which there is an edge from state s on input c. The function *nextChar* returns the next character of the input string x. □

Example 3.19: In Fig. 3.28 we see the transition graph of a DFA accepting the language $\mathbf{(a|b)^*abb}$, the same as that accepted by the NFA of Fig. 3.24. Given the input string *ababb*, this DFA enters the sequence of states $0, 1, 2, 1, 2, 3$ and returns "yes." □

```
s = s0;
c = nextChar();
while ( c != eof ) {
        s = move(s, c);
        c = nextChar();
}
if ( s is in F ) return "yes";
else return "no";
```

Figure 3.27: Simulating a DFA

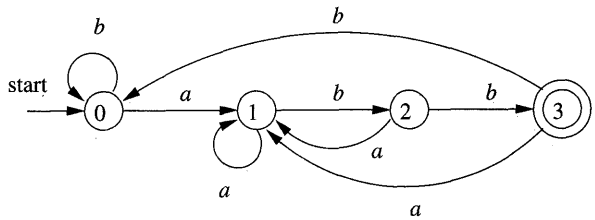


Figure 3.28: DFA accepting (**a**|**b**)*__abb__

3.6.5 Exercises for Section 3.6

! **Exercise 3.6.1:** Figure 3.19 in the exercises of Section 3.4 computes the failure function for the KMP algorithm. Show how, given that failure function, we can construct, from a keyword $b_1 b_2 \cdots b_n$ an $n+1$-state DFA that recognizes $.^* b_1 b_2 \cdots b_n$, where the dot stands for "any character." Moreover, this DFA can be constructed in $O(n)$ time.

Exercise 3.6.2: Design finite automata (deterministic or nondeterministic) for each of the languages of Exercise 3.3.5.

Exercise 3.6.3: For the NFA of Fig. 3.29, indicate all the paths labeled *aabb*. Does the NFA accept *aabb*?

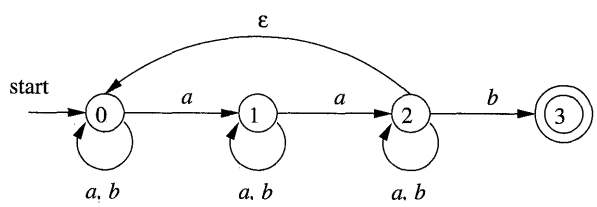


Figure 3.29: NFA for Exercise 3.6.3

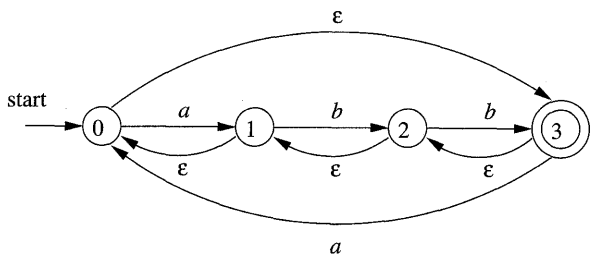


Figure 3.30: NFA for Exercise 3.6.4

Exercise 3.6.4: Repeat Exercise 3.6.3 for the NFA of Fig. 3.30.

Exercise 3.6.5: Give the transition tables for the NFA of:

a) Exercise 3.6.3.

b) Exercise 3.6.4.

c) Figure 3.26.

3.7 From Regular Expressions to Automata

The regular expression is the notation of choice for describing lexical analyzers and other pattern-processing software, as was reflected in Section 3.5. However, implementation of that software requires the simulation of a DFA, as in Algorithm 3.18, or perhaps simulation of an NFA. Because an NFA often has a choice of move on an input symbol (as Fig. 3.24 does on input a from state 0) or on ϵ (as Fig. 3.26 does from state 0), or even a choice of making a transition on ϵ or on a real input symbol, its simulation is less straightforward than for a DFA. Thus often it is important to convert an NFA to a DFA that accepts the same language.

In this section we shall first show how to convert NFA's to DFA's. Then, we use this technique, known as "the subset construction," to give a useful algorithm for simulating NFA's directly, in situations (other than lexical analysis) where the NFA-to-DFA conversion takes more time than the direct simulation. Next, we show how to convert regular expressions to NFA's, from which a DFA can be constructed if desired. We conclude with a discussion of the time-space tradeoffs inherent in the various methods for implementing regular expressions, and see how to choose the appropriate method for your application.

3.7.1 Conversion of an NFA to a DFA

The general idea behind the subset construction is that each state of the constructed DFA corresponds to a set of NFA states. After reading input

$a_1 a_2 \cdots a_n$, the DFA is in that state which corresponds to the set of states that the NFA can reach, from its start state, following paths labeled $a_1 a_2 \cdots a_n$.

It is possible that the number of DFA states is exponential in the number of NFA states, which could lead to difficulties when we try to implement this DFA. However, part of the power of the automaton-based approach to lexical analysis is that for real languages, the NFA and DFA have approximately the same number of states, and the exponential behavior is not seen.

Algorithm 3.20: The *subset construction* of a DFA from an NFA.

INPUT: An NFA N.

OUTPUT: A DFA D accepting the same language as N.

METHOD: Our algorithm constructs a transition table *Dtran* for D. Each state of D is a set of NFA states, and we construct *Dtran* so D will simulate "in parallel" all possible moves N can make on a given input string. Our first problem is to deal with ϵ-transitions of N properly. In Fig. 3.31 we see the definitions of several functions that describe basic computations on the states of N that are needed in the algorithm. Note that s is a single state of N, while T is a set of states of N.

OPERATION	DESCRIPTION
ϵ-*closure*(s)	Set of NFA states reachable from NFA state s on ϵ-transitions alone.
ϵ-*closure*(T)	Set of NFA states reachable from some NFA state s in set T on ϵ-transitions alone; $= \cup_{s \text{ in } T}$ ϵ-*closure*(s).
move(T, a)	Set of NFA states to which there is a transition on input symbol a from some state s in T.

Figure 3.31: Operations on NFA states

We must explore those sets of states that N can be in after seeing some input string. As a basis, before reading the first input symbol, N can be in any of the states of ϵ-*closure*(s_0), where s_0 is its start state. For the induction, suppose that N can be in set of states T after reading input string x. If it next reads input a, then N can immediately go to any of the states in *move*(T, a). However, after reading a, it may also make several ϵ-transitions; thus N could be in any state of ϵ-*closure*(*move*(T, a)) after reading input xa. Following these ideas, the construction of the set of D's states, *Dstates*, and its transition function *Dtran*, is shown in Fig. 3.32.

The start state of D is ϵ-*closure*(s_0), and the accepting states of D are all those sets of N's states that include at least one accepting state of N. To complete our description of the subset construction, we need only to show how

```
initially, ε-closure(s0) is the only state in Dstates, and it is unmarked;
while ( there is an unmarked state T in Dstates ) {
        mark T;
        for ( each input symbol a ) {
                U = ε-closure(move(T, a));
                if ( U is not in Dstates )
                        add U as an unmarked state to Dstates;
                Dtran[T, a] = U;
        }
}
```

Figure 3.32: The subset construction

ϵ-*closure*(T) is computed for any set of NFA states T. This process, shown in Fig. 3.33, is a straightforward search in a graph from a set of states. In this case, imagine that only the ϵ-labeled edges are available in the graph. □

```
push all states of T onto stack;
initialize ε-closure(T) to T;
while ( stack is not empty ) {
        pop t, the top element, off stack;
        for ( each state u with an edge from t to u labeled ε )
                if ( u is not in ε-closure(T) ) {
                        add u to ε-closure(T);
                        push u onto stack;
                }
}
```

Figure 3.33: Computing ϵ-*closure*(T)

Example 3.21: Figure 3.34 shows another NFA accepting (**a**|**b**)***abb**; it happens to be the one we shall construct directly from this regular expression in Section 3.7. Let us apply Algorithm 3.20 to Fig. 3.29.

The start state A of the equivalent DFA is ϵ-*closure*(0), or $A = \{0, 1, 2, 4, 7\}$, since these are exactly the states reachable from state 0 via a path all of whose edges have label ϵ. Note that a path can have zero edges, so state 0 is reachable from itself by an ϵ-labeled path.

The input alphabet is $\{a, b\}$. Thus, our first step is to mark A and compute $Dtran[A, a] = \epsilon\text{-}closure(move(A, a))$ and $Dtran[A, b] = \epsilon\text{-}closure(move(A, b))$. Among the states 0, 1, 2, 4, and 7, only 2 and 7 have transitions on a, to 3 and 8, respectively. Thus, $move(A, a) = \{3, 8\}$. Also, $\epsilon\text{-}closure(\{3, 8\} = \{1, 2, 3, 4, 6, 7, 8\}$, so we conclude

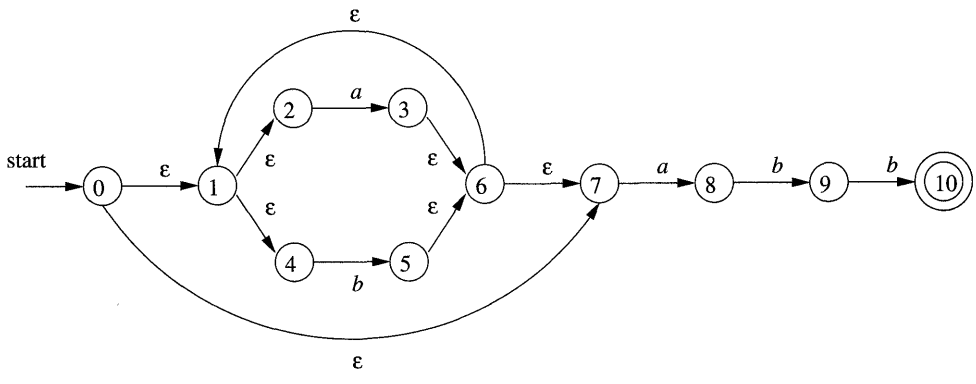


Figure 3.34: NFA N for **(a|b)*abb**

$$Dtran[A, a] = \epsilon\text{-}closure\big(move(A, a)\big) = \epsilon\text{-}closure(\{3, 8\}) = \{1, 2, 3, 4, 6, 7, 8\}$$

Let us call this set B, so $Dtran[A, a] = B$.

Now, we must compute $Dtran[A, b]$. Among the states in A, only 4 has a transition on b, and it goes to 5. Thus,

$$Dtran[A, b] = \epsilon\text{-}closure(\{5\}) = \{1, 2, 4, 6, 7\}$$

Let us call the above set C, so $Dtran[A, b] = C$.

NFA STATE	DFA STATE	a	b
$\{0, 1, 2, 4, 7\}$	A	B	C
$\{1, 2, 3, 4, 6, 7, 8\}$	B	B	D
$\{1, 2, 4, 5, 6, 7\}$	C	B	C
$\{1, 2, 4, 5, 6, 7, 9\}$	D	B	E
$\{1, 2, 3, 5, 6, 7, 10\}$	E	B	C

Figure 3.35: Transition table *Dtran* for DFA D

If we continue this process with the unmarked sets B and C, we eventually reach a point where all the states of the DFA are marked. This conclusion is guaranteed, since there are "only" 2^{11} different subsets of a set of eleven NFA states. The five different DFA states we actually construct, their corresponding sets of NFA states, and the transition table for the DFA D are shown in Fig. 3.35, and the transition graph for D is in Fig. 3.36. State A is the start state, and state E, which contains state 10 of the NFA, is the only accepting state.

Note that D has one more state than the DFA of Fig. 3.28 for the same language. States A and C have the same move function, and so can be merged. We discuss the matter of minimizing the number of states of a DFA in Section 3.9.6. □

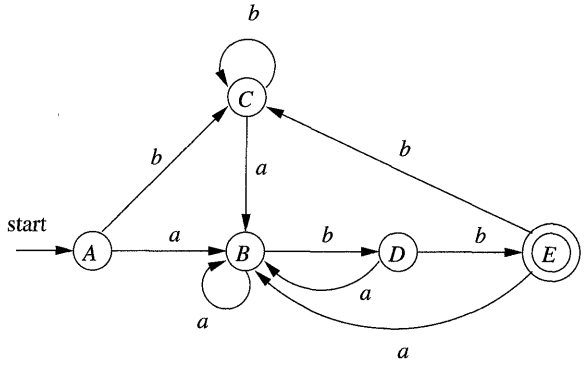


Figure 3.36: Result of applying the subset construction to Fig. 3.34

3.7.2 Simulation of an NFA

A strategy that has been used in a number of text-editing programs is to construct an NFA from a regular expression and then simulate the NFA using something like an on-the-fly subset construction. The simulation is outlined below.

Algorithm 3.22: Simulating an NFA.

INPUT: An input string x terminated by an end-of-file character **eof**. An NFA N with start state s_0, accepting states F, and transition function *move*.

OUTPUT: Answer "yes" if M accepts x; "no" otherwise.

METHOD: The algorithm keeps a set of current states S, those that are reached from s_0 following a path labeled by the inputs read so far. If c is the next input character, read by the function *nextChar*(), then we first compute $move(S, c)$ and then close that set using ϵ-*closure*(). The algorithm is sketched in Fig. 3.37. □

```
1)  S = ε-closure(s0);
2)  c = nextChar();
3)  while ( c != eof ) {
4)          S = ε-closure(move(S, c));
5)          c = nextChar();
6)  }
7)  if ( S ∩ F != ∅ ) return "yes";
8)  else return "no";
```

Figure 3.37: Simulating an NFA

3.7.3 Efficiency of NFA Simulation

If carefully implemented, Algorithm 3.22 can be quite efficient. As the ideas involved are useful in a number of similar algorithms involving search of graphs, we shall look at this implementation in additional detail. The data structures we need are:

1. Two stacks, each of which holds a set of NFA states. One of these stacks, *oldStates*, holds the "current" set of states, i.e., the value of S on the right side of line (4) in Fig. 3.37. The second, *newStates*, holds the "next" set of states — S on the left side of line (4). Unseen is a step where, as we go around the loop of lines (3) through (6), *newStates* is transferred to *oldStates*.

2. A boolean array *alreadyOn*, indexed by the NFA states, to indicate which states are in *newStates*. While the array and stack hold the same information, it is much faster to interrogate *alreadyOn*$[s]$ than to search for state s on the stack *newStates*. It is for this efficiency that we maintain both representations.

3. A two-dimensional array *move*$[s, a]$ holding the transition table of the NFA. The entries in this table, which are sets of states, are represented by linked lists.

To implement line (1) of Fig. 3.37, we need to set each entry in array *alreadyOn* to `FALSE`, then for each state s in ϵ-*closure*(s_0), push s onto *oldStates* and set *alreadyOn*$[s]$ to `TRUE`. This operation on state s, and the implementation of line (4) as well, are facilitated by a function we shall call *addState*(s). This function pushes state s onto *newStates*, sets *alreadyOn*$[s]$ to `TRUE`, and calls itself recursively on the states in *move*$[s, \epsilon]$ in order to further the computation of ϵ-*closure*(s). However, to avoid duplicating work, we must be careful never to call *addState* on a state that is already on the stack *newStates*. Figure 3.38 sketches this function.

```
 9)  addState(s) {
10)        push s onto newStates;
11)        alreadyOn[s] = TRUE;
12)        for ( t on move[s, ε] )
13)              if ( !alreadyOn(t) )
14)                    addState(t);
15)  }
```

Figure 3.38: Adding a new state s, which is known not to be on *newStates*

We implement line (4) of Fig. 3.37 by looking at each state s on *oldStates*. We first find the set of states *move*$[s, c]$, where c is the next input, and for each

of those states that is not already on *newStates*, we apply *addState* to it. Note that *addState* has the effect of computing the ϵ-*closure* and adding all those states to *newStates* as well, if they were not already on. This sequence of steps is summarized in Fig. 3.39.

```
16)  for ( s on oldStates ) {
17)      for ( t on move[s, c] )
18)          if ( !alreadyOn[t] )
19)              addState(t);
20)      pop s from oldStates;
21)  }

22)  for ( s on newStates ) {
23)      pop s from newStates;
24)      push s onto oldStates;
25)      alreadyOn[s] = FALSE;
26)  }
```

Figure 3.39: Implementation of step (4) of Fig. 3.37

Now, suppose that the NFA N has n states and m transitions; i.e., m is the sum over all states of the number of symbols (or ϵ) on which the state has a transition out. Not counting the call to *addState* at line (19) of Fig. 3.39, the time spent in the loop of lines (16) through (21) is $O(n)$. That is, we can go around the loop at most n times, and each step of the loop requires constant work, except for the time spent in *addState*. The same is true of the loop of lines (22) through (26).

During one execution of Fig. 3.39, i.e., of step (4) of Fig. 3.37, it is only possible to call *addState* on a given state once. The reason is that whenever we call *addState*(s), we set *alreadyOn*[s] to TRUE at line (11) of Fig. 3.39. Once *alreadyOn*[s] is TRUE, the tests at line (13) of Fig. 3.38 and line (18) of Fig. 3.39 prevent another call.

The time spent in one call to *addState*, exclusive of the time spent in recursive calls at line (14), is $O(1)$ for lines (10) and (11). For lines (12) and (13), the time depends on how many ϵ-transitions there are out of state s. We do not know this number for a given state, but we know that there are at most m transitions in total, out of all states. As a result, the aggregate time spent in lines (11) over all calls to *addState* during one execution of the code of Fig. 3.39 is $O(m)$. The aggregate for the rest of the steps of *addState* is $O(n)$, since it is a constant per call, and there are at most n calls.

We conclude that, implemented properly, the time to execute line (4) of Fig. 3.37 is $O(n + m)$. The rest of the while-loop of lines (3) through (6) takes $O(1)$ time per iteration. If the input x is of length k, then the total work in that loop is $O\big(k(n + m)\big)$. Line (1) of Fig. 3.37 can be executed in $O(n + m)$ time, since it is essentially the steps of Fig. 3.39 with *oldStates* containing only

Big-Oh Notation

An expression like $O(n)$ is a shorthand for "at most some constant times n." Technically, we say a function $f(n)$, perhaps the running time of some step of an algorithm, is $O(g(n))$ if there are constants c and n_0, such that whenever $n \geq n_0$, it is true that $f(n) \leq cg(n)$. A useful idiom is "$O(1)$," which means "some constant." The use of this *big-oh notation* enables us to avoid getting too far into the details of what we count as a unit of execution time, yet lets us express the rate at which the running time of an algorithm grows.

the state s_0. Lines (2), (7), and (8) each take $O(1)$ time. Thus, the running time of Algorithm 3.22, properly implemented, is $O\big(\big(k(n+m)\big)\big)$. That is, the time taken is proportional to the length of the input times the size (nodes plus edges) of the transition graph.

3.7.4 Construction of an NFA from a Regular Expression

We now give an algorithm for converting any regular expression to an NFA that defines the same language. The algorithm is syntax-directed, in the sense that it works recursively up the parse tree for the regular expression. For each subexpression the algorithm constructs an NFA with a single accepting state.

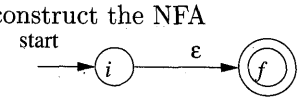
Algorithm 3.23: The McNaughton-Yamada-Thompson algorithm to convert a regular expression to an NFA.

INPUT: A regular expression r over alphabet Σ.

OUTPUT: An NFA N accepting $L(r)$.

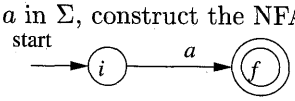
METHOD: Begin by parsing r into its constituent subexpressions. The rules for constructing an NFA consist of basis rules for handling subexpressions with no operators, and inductive rules for constructing larger NFA's from the NFA's for the immediate subexpressions of a given expression.

BASIS: For expression ϵ construct the NFA



Here, i is a new state, the start state of this NFA, and f is another new state, the accepting state for the NFA.

For any subexpression a in Σ, construct the NFA



where again i and f are new states, the start and accepting states, respectively. Note that in both of the basis constructions, we construct a distinct NFA, with new states, for every occurrence of ϵ or some a as a subexpression of r.

INDUCTION: Suppose $N(s)$ and $N(t)$ are NFA's for regular expressions s and t, respectively.

a) Suppose $r = s|t$. Then $N(r)$, the NFA for r, is constructed as in Fig. 3.40. Here, i and f are new states, the start and accepting states of $N(r)$, respectively. There are ϵ-transitions from i to the start states of $N(s)$ and $N(t)$, and each of their accepting states have ϵ-transitions to the accepting state f. Note that the accepting states of $N(s)$ and $N(t)$ are not accepting in $N(r)$. Since any path from i to f must pass through either $N(s)$ or $N(t)$ exclusively, and since the label of that path is not changed by the ϵ's leaving i or entering f, we conclude that $N(r)$ accepts $L(s) \cup L(t)$, which is the same as $L(r)$. That is, Fig. 3.40 is a correct construction for the union operator.

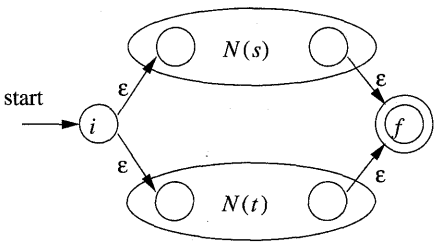


Figure 3.40: NFA for the union of two regular expressions

b) Suppose $r = st$. Then construct $N(r)$ as in Fig. 3.41. The start state of $N(s)$ becomes the start state of $N(r)$, and the accepting state of $N(t)$ is the only accepting state of $N(r)$. The accepting state of $N(s)$ and the start state of $N(t)$ are merged into a single state, with all the transitions in or out of either state. A path from i to f in Fig. 3.41 must go first through $N(s)$, and therefore its label will begin with some string in $L(s)$. The path then continues through $N(t)$, so the path's label finishes with a string in $L(t)$. As we shall soon argue, accepting states never have edges out and start states never have edges in, so it is not possible for a path to re-enter $N(s)$ after leaving it. Thus, $N(r)$ accepts exactly $L(s)L(t)$, and is a correct NFA for $r = st$.

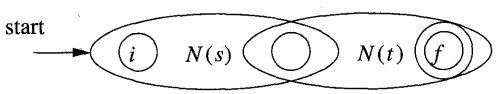


Figure 3.41: NFA for the concatenation of two regular expressions

c) Suppose $r = s^*$. Then for r we construct the NFA $N(r)$ shown in Fig. 3.42. Here, i and f are new states, the start state and lone accepting state of $N(r)$. To get from i to f, we can either follow the introduced path labeled ϵ, which takes care of the one string in $L(s)^0$, or we can go to the start state of $N(s)$, through that NFA, then from its accepting state back to its start state zero or more times. These options allow $N(r)$ to accept all the strings in $L(s)^1$, $L(s)^2$, and so on, so the entire set of strings accepted by $N(r)$ is $L(s^*)$.

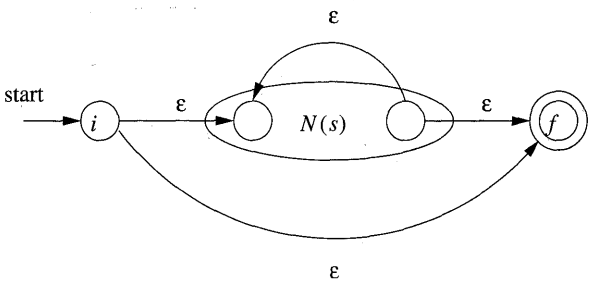


Figure 3.42: NFA for the closure of a regular expression

d) Finally, suppose $r = (s)$. Then $L(r) = L(s)$, and we can use the NFA $N(s)$ as $N(r)$.

□

The method description in Algorithm 3.23 contains hints as to why the inductive construction works as it should. We shall not give a formal correctness proof, but we shall list several properties of the constructed NFA's, in addition to the all-important fact that $N(r)$ accepts language $L(r)$. These properties are interesting in their own right, and helpful in making a formal proof.

1. $N(r)$ has at most twice as many states as there are operators and operands in r. This bound follows from the fact that each step of the algorithm creates at most two new states.

2. $N(r)$ has one start state and one accepting state. The accepting state has no outgoing transitions, and the start state has no incoming transitions.

3. Each state of $N(r)$ other than the accepting state has either one outgoing transition on a symbol in Σ or two outgoing transitions, both on ϵ.

Example 3.24: Let us use Algorithm 3.23 to construct an NFA for $r = (\mathbf{a}|\mathbf{b})^*\mathbf{abb}$. Figure 3.43 shows a parse tree for r that is analogous to the parse trees constructed for arithmetic expressions in Section 2.2.3. For subexpression r_1, the first **a**, we construct the NFA:

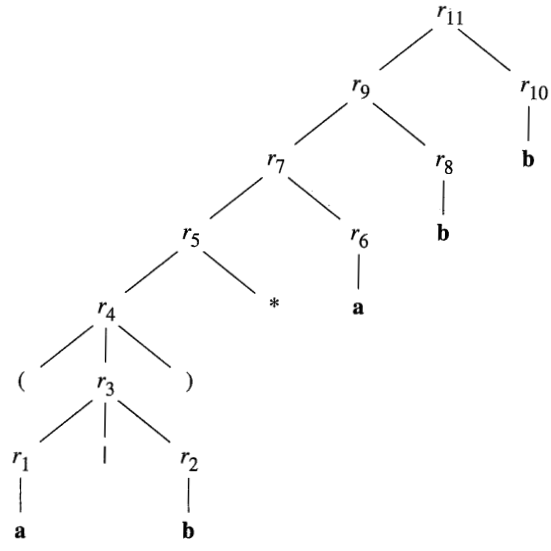
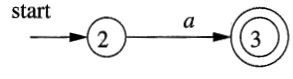
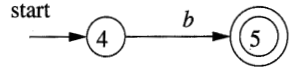


Figure 3.43: Parse tree for (**a**|**b**)***abb**



State numbers have been chosen for consistency with what follows. For r_2 we construct:



We can now combine $N(r_1)$ and $N(r_2)$, using the construction of Fig. 3.40 to obtain the NFA for $r_3 = r_1|r_2$; this NFA is shown in Fig. 3.44.

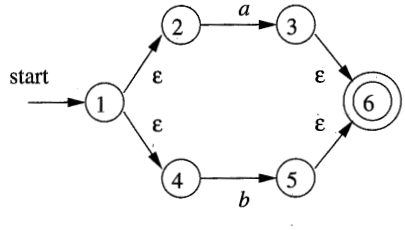


Figure 3.44: NFA for r_3

The NFA for $r_4 = (r_3)$ is the same as that for r_3. The NFA for $r_5 = (r_3)^*$ is then as shown in Fig. 3.45. We have used the construction in Fig. 3.42 to build this NFA from the NFA in Fig. 3.44.

Now, consider subexpression r_6, which is another **a**. We use the basis construction for a again, but we must use new states. It is not permissible to reuse

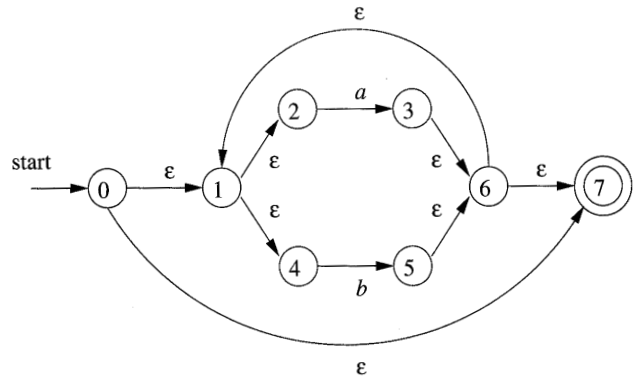
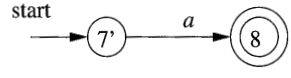


Figure 3.45: NFA for r_5

the NFA we constructed for r_1, even though r_1 and r_6 are the same expression. The NFA for r_6 is:



To obtain the NFA for $r_7 = r_5 r_6$, we apply the construction of Fig. 3.41. We merge states 7 and 7', yielding the NFA of Fig. 3.46. Continuing in this fashion with new NFA's for the two subexpressions **b** called r_8 and r_{10}, we eventually construct the NFA for **(a|b)*abb** that we first met in Fig. 3.34. □

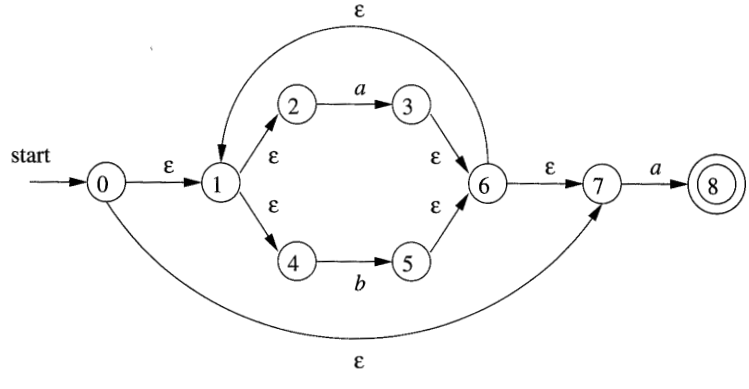


Figure 3.46: NFA for r_7

3.7.5 Efficiency of String-Processing Algorithms

We observed that Algorithm 3.18 processes a string x in time $O(|x|)$, while in Section 3.7.3 we concluded that we could simulate an NFA in time proportional to the product of $|x|$ and the size of the NFA's transition graph. Obviously, it

is faster to have a DFA to simulate than an NFA, so we might wonder whether it ever makes sense to simulate an NFA.

One issue that may favor an NFA is that the subset construction can, in the worst case, exponentiate the number of states. While in principle, the number of DFA states does not influence the running time of Algorithm 3.18, should the number of states become so large that the transition table does not fit in main memory, then the true running time would have to include disk I/O and therefore rise noticeably.

Example 3.25: Consider the family of languages described by regular expressions of the form $L_n = (\mathbf{a}|\mathbf{b})^*\mathbf{a}(\mathbf{a}|\mathbf{b})^{n-1}$, that is, each language L_n consists of strings of a's and b's such that the nth character to the left of the right end holds a. An $n+1$-state NFA is easy to construct. It stays in its initial state under any input, but also has the option, on input a, of going to state 1. From state 1, it goes to state 2 on any input, and so on, until in state n it accepts. Figure 3.47 suggests this NFA.

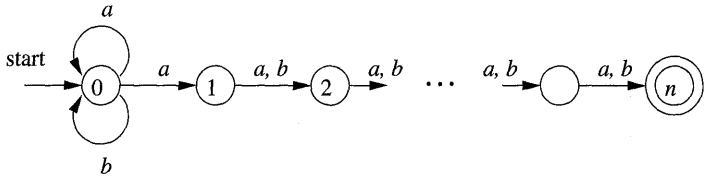


Figure 3.47: An NFA that has many fewer states than the smallest equivalent DFA

However, any DFA for the language L_n must have at least 2^n states. We shall not prove this fact, but the idea is that if two strings of length n can get the DFA to the same state, then we can exploit the last position where the strings differ (and therefore one must have a, the other b) to continue the strings identically, until they are the same in the last $n-1$ positions. The DFA will then be in a state where it must both accept and not accept. Fortunately, as we mentioned, it is rare for lexical analysis to involve patterns of this type, and we do not expect to encounter DFA's with outlandish numbers of states in practice. □

However, lexical-analyzer generators and other string-processing systems often start with a regular expression. We are faced with a choice of converting the regular expression to an NFA or DFA. The additional cost of going to a DFA is thus the cost of executing Algorithm 3.23 on the NFA (one could go directly from a regular expression to a DFA, but the work is essentially the same). If the string-processor is one that will be executed many times, as is the case for lexical analysis, then any cost of converting to a DFA is worthwhile. However, in other string-processing applications, such as `grep`, where the user specifies one regular expression and one or several files to be searched for the pattern

of that expression, it may be more efficient to skip the step of constructing a DFA, and simulate the NFA directly.

Let us consider the cost of converting a regular expression r to an NFA by Algorithm 3.23. A key step is constructing the parse tree for r. In Chapter 4 we shall see several methods that are capable of constructing this parse tree in linear time, that is, in time $O(|r|)$, where $|r|$ stands for the *size* of r — the sum of the number of operators and operands in r. It is also easy to check that each of the basis and inductive constructions of Algorithm 3.23 takes constant time, so the entire time spent by the conversion to an NFA is $O(|r|)$.

Moreover, as we observed in Section 3.7.4, the NFA we construct has at most $|r|$ states and at most $2|r|$ transitions. That is, in terms of the analysis in Section 3.7.3, we have $n \leq |r|$ and $m \leq 2|r|$. Thus, simulating this NFA on an input string x takes time $O(|r| \times |x|)$. This time dominates the time taken by the NFA construction, which is $O(|r|)$, and therefore, we conclude that it is possible to take a regular expression r and string x, and tell whether x is in $L(r)$ in time $O(|r| \times |x|)$.

The time taken by the subset construction is highly dependent on the number of states the resulting DFA has. To begin, notice that in the subset construction of Fig. 3.32, the key step, the construction of a set of states U from a set of states T and an input symbol a, is very much like the construction of a new set of states from the old set of states in the NFA simulation of Algorithm 3.22. We already concluded that, properly implemented, this step takes time at most proportional to the number of states and transitions of the NFA.

Suppose we start with a regular expression r and convert it to an NFA. This NFA has at most $|r|$ states and at most $2|r|$ transitions. Moreover, there are at most $|r|$ input symbols. Thus, for every DFA state constructed, we must construct at most $|r|$ new states, and each one takes at most $O(|r| + 2|r|)$ time. The time to construct a DFA of s states is thus $O(|r|^2 s)$.

In the common case where s is about $|r|$, the subset construction takes time $O(|r|^3)$. However, in the worst case, as in Example 3.25, this time is $O(|r|^2 2^{|r|})$. Figure 3.48 summarizes the options when one is given a regular expression r and wants to produce a recognizer that will tell whether one or more strings x are in $L(r)$.

AUTOMATON	INITIAL	PER STRING
NFA	$O(\|r\|)$	$O(\|r\| \times \|x\|)$
DFA typical case	$O(\|r\|^3)$	$O(\|x\|)$
DFA worst case	$O(\|r\|^2 2^{\|r\|})$	$O(\|x\|)$

Figure 3.48: Initial cost and per-string-cost of various methods of recognizing the language of a regular expression

If the per-string cost dominates, as it does when we build a lexical analyzer,

we clearly prefer the DFA. However, in commands like `grep`, where we run the automaton on only one string, we generally prefer the NFA. It is not until $|x|$ approaches $|r|^3$ that we would even think about converting to a DFA.

There is, however, a mixed strategy that is about as good as the better of the NFA and the DFA strategy for each expression r and string x. Start off simulating the NFA, but remember the sets of NFA states (i.e., the DFA states) and their transitions, as we compute them. Before processing the current set of NFA states and the current input symbol, check to see whether we have already computed this transition, and use the information if so.

3.7.6 Exercises for Section 3.7

Exercise 3.7.1: Convert to DFA's the NFA's of:

a) Fig. 3.26.

b) Fig. 3.29.

c) Fig. 3.30.

Exercise 3.7.2: use Algorithm 3.22 to simulate the NFA's:

a) Fig. 3.29.

b) Fig. 3.30.

on input *aabb*.

Exercise 3.7.3: Convert the following regular expressions to deterministic finite automata, using algorithms 3.23 and 3.20:

a) $(\mathbf{a}|\mathbf{b})^*$.

b) $(\mathbf{a}^*|\mathbf{b}^*)^*$.

c) $\left((\epsilon|\mathbf{a})\mathbf{b}^*\right)^*$.

d) $(\mathbf{a}|\mathbf{b})^*\mathbf{abb}(\mathbf{a}|\mathbf{b})^*$.

3.8 Design of a Lexical-Analyzer Generator

In this section we shall apply the techniques presented in Section 3.7 to see how a lexical-analyzer generator such as `Lex` is architected. We discuss two approaches, based on NFA's and DFA's; the latter is essentially the implementation of `Lex`.